Rapport Projet INFO 7

Nom de l'étudiant : Mahamat Alhafiz
Prénom de l'étudiant : Kerim Andoulaye
Nom de l'étudiant 2 :mhammed Lalou

Encadrant :M. Jean Fromentin

Année universitaire :2024–2025

Introduction

Jeu : Snake - Trié

Le projet est divisé en 4 séances, et ce rapport détaille le rendu de la dernière séance, qui correspond à la phase finale du jeu.

Les fichiers du projet sont organisés comme suit :

1. **snake.cpp et snake.hpp** : Ces fichiers sont spécifiquement dédiés à la gestion du serpent dans le jeu. Ils contiennent les fonctions pour la création, le déplacement, et l'évolution du serpent.
2. **gestionEvent.cpp et gestionEvent.hpp** : Ces fichiers sont utilisés pour la gestion des événements du jeu, comme la détection des entrées clavier (par exemple, pour les déplacements du serpent).
3. **window.cpp et window.hpp** : Ces fichiers sont responsables de l'affichage de la fenêtre du jeu. Ils gèrent la création de la fenêtre, l'affichage du jeu et l'interface utilisateur.
4. **world.cpp et world.hpp** : Ces fichiers sont utilisés pour l'initialisation du monde du jeu, y compris la gestion de la nourriture et des obstacles dans le jeu.
5. **enum.hpp** : Ce fichier contient l'énumération des différents types de nourriture dans le jeu, tels que la nourriture verte, rouge, bleue et étoilée.
6. **view.cpp et view.hpp** : Ces fichiers gèrent l'affichage du serpent et du monde, en utilisant les images correspondantes pour rendre le jeu plus visuel et interactif.
7. **game.cpp et game.hpp** : Ces fichiers contiennent la logique principale du jeu, où toutes les actions et interactions (comme les collisions, les mouvements, la gestion des scores, etc.) sont traitées.

Explication du fichier world.cpp

Le fichier world.cpp contient la logique pour gérer le monde du jeu, y compris l'initialisation, la lecture, l'écriture, et l'ajout de la nourriture. Voici une explication détaillée des principales fonctions :

1. void init_world(World* world, int w, int h) :
   Cette fonction initialise le monde du jeu avec des cases de type Empty. Elle crée un monde de dimensions spécifiées par les paramètres w (largeur) et h (hauteur).
2. void free_world(World* world) :
   Cette fonction est utilisée pour libérer les ressources allouées pour le monde du jeu. Elle détruit toutes les données liées au monde et permet de libérer la mémoire associée.

3. Block read(World* world, int x, int y) :
   Cette fonction permet de lire l'état de la case située aux coordonnées (x, y) dans le monde. Elle retourne le type de bloc (par exemple, vide,redfood) situé à ces coordonnées.
4. void write(World* world, int x, int y, Block b) :
   Cette fonction permet d'écrire un nouveau bloc dans le monde, à la position (x, y). Le bloc peut être de différents types, comme un vide,une nourriture.
5. void add_food(World* world, int x, int y, Block type) :
   Cette fonction permet d'ajouter de la nourriture dans le monde à une position (x, y), avec un type spécifique (par exemple, nourriture verte, rouge, bleue ou étoilée). Elle utilise la fonction write pour insérer la nourriture dans le monde à l'endroit spécifié.
6. void init_food(World* world, int number_food) :
   Cette fonction gère l'ajout de 4 types différents de nourriture (avec des couleurs distinctes) à des positions aléatoires dans le monde. Elle appelle la fonction read pour vérifier si une case est vide avant d'y placer de la nourriture, et utilise add_food pour insérer la nourriture dans le monde.

Ces fonctions sont essentielles pour la gestion de l'environnement de jeu, assurant que le monde soit correctement initialisé, que les éléments (comme la nourriture) soient ajoutés à des positions aléatoires, et que le serpent puisse interagir avec ces éléments.

Explication du fichier snake.cpp

Le fichier snake.cpp contient la logique principale liée au serpent : son initialisation, son mouvement, et la gestion des anneaux qui forment son corps. Voici une explication détaillée des fonctions principales :

1. void init_snake(Snake* snake, int x, int y, Direction dir_initiale) :
   Cette fonction initialise le serpent au début du jeu. Elle place la tête du serpent aux coordonnées (x, y) au milieu et lui assigne une direction initiale (comme North, South, East, ou West). Cette fonction sert à démarrer le jeu avec un serpent de taille minimale (un seul anneau, la tête).

Explication du fichier enum.hpp

Le fichier enum.hpp contient des énumérations qui définissent les différentes directions que peut prendre le serpent, ainsi que les types de blocs qui peuvent apparaître dans le monde (comme les nourritures).

1. enum Direction { North, South, East, West } :
   Cette énumération définit les différentes directions que peut prendre le serpent. Les valeurs possibles sont :
   - North (Nord) : La tête du serpent se déplace vers le haut.
   - South (Sud) : La tête du serpent se déplace vers le bas.

- East (Est) : La tête du serpent se déplace vers la droite.
- West (Ouest) : La tête du serpent se déplace vers la gauche.

2. enum Block { Empty, RedFood, GreenFood, BlueFood, StarFood } :
   Cette énumération définit les différents types de blocs qui peuvent se trouver dans le monde du jeu :
   - Empty : Un espace vide.
   - RedFood : Nourriture rouge.
   - GreenFood : Nourriture verte.
   - BlueFood : Nourriture bleue.
   - StarFood : Nourriture étoilée (peut-être une nourriture spéciale ou donnant un bonus).

Explication du fichier gestionevent.cpp

La fonction principale de ce fichier est :

bool handle_event(Game *game, string filename) ;

Rôle de la fonction handle_event :

Cette fonction gère **les événements clavier** pendant le jeu, c'est-à-dire les actions que le joueur effectue avec le clavier. Elle permet de :

- Lire les touches directionnelles pour **changer la direction du serpent** :
  - **Flèche haut** → aller vers le haut
  - **Flèche bas** → aller vers le bas
  - **Flèche gauche** → aller à gauche
  - **Flèche droite** → aller à droite
- Gérer des **touches spéciales** :
  - Si on appuie sur q, le jeu s'arrête.
  - Si on appuie sur s, le jeu est **sauvegardé** :
    - La fonction appelle save_game(game, filename) pour enregistrer la partie.
    - Ensuite, une pause s'affiche :

      "Partie sauvegardée. Voulez-vous continuer ? (o/n)"

    - Si le joueur appuie sur o, le jeu continue.
    - Si le joueur appuie sur n, le jeu s'arrête

Résumé :

- **handle_event** gère les **touches du clavier** pour contrôler le serpent et exécuter des actions importantes (sauvegarder, quitter, changer de direction).
- Elle utilise la fonction save_game (déjà définie dans game.cpp) pour faire la sauvegarde.
- L'interaction avec l'utilisateur se fait via le **clavier**, avec des confirmations après certaines actions.

Explication du fichier view.cpp

Le fichier view.cpp contient les fonctions **d'affichage du jeu** : c'est ici qu'on dessine le **monde**, le **score**, la **nourriture**, et le **serpent**.

Rôle :

Cette fonction **affiche le fond du jeu**, le **score**, et les **éléments du monde (nourriture)**.

Étapes :

1. **Affichage de l'image de fond** (background) :

draw_texture(window, view->background, 0, 0, window->width, window→height);

0, 0 = coin supérieur gauche.

- Couvre toute la fenêtre

Affichage du score :

string score_text = "Score: " + to_string(score);

set_color(&window->background, 255, 255, 255, 255); // blanc

set_color(&window->foreground, 12, 13, 45, 66); // texte

draw_text(window, score_text, 1, 1); // position haut gauche

Parcours de la grille du monde :

Utilise la fonction read() pour lire le contenu de chaque case.

- Si la case contient de la nourriture (RedFood, GreenFood, etc.), affiche la texture correspondante.

void draw_snake(Window* window, Snake* snake, SnakeView* snakeview, World* world)

Rôle :

Affiche **le corps du serpent** (les anneaux) et **sa tête**, avec les **bonnes textures selon la couleur et la direction**.

Étapes :

1. **Afficher les anneaux du corps** :

- Parcours la liste chaînée des anneaux (Anneau* current = snake->first)
- Pour chaque anneau, affiche la texture en fonction de sa couleur (val) :

2-Afficher la tête du serpent :

- Selon la **direction du serpent** (snake->diremplacement)
- Si la bouche est **ouverte** (snake->mouth_open == true) → afficher une texture "open"
- Sinon, afficher la texture "close"
  -

Problème rencontré :

J' avais un bug au début car je dessinais les anneaux *après* la tête, donc l'image de la tête était cachée.
Ja i corrigé cela en affichant d'abord le corps, puis la tête à la fin.

void init_view(Window* window, Game* game)

Cette fonction initialise l'affichage du jeu :

- Elle **alloue dynamiquement** la structure WorldView.
- Charge les **textures des nourritures** avec la fonction load_image(window, nomFichier) selon leur type.

  . ensuite la lloue SnakeView, qui contient :
  - Les textures de la **tête du serpent** dans toutes les directions (ouverte et fermée).
  - Les textures du **corps** du serpent pour chaque couleur.

bool move_snake(Game* game)

C'est la fonction principale qui gère le **mouvement du serpent** :

- Elle appelle move_anneau(game->snake, game->world) pour faire suivre les anneaux à la tête.
- Ensuite, elle met à jour la **position de la tête** selon sa direction :
  - Vers le haut : on diminue y.
  - Vers le bas : on augmente y.
  - Vers la gauche : on diminue x.
  - Vers la droite : on augmente x.
- Elle utilise write() pour **dessiner la tête à sa nouvelle position**.

- Elle possede le passage secret en bas a gauche et le serpent sort en haut a droite
- elle possède le rébondissemnt en haut au milieu, le serpent rebondis
- Elle vérifie ensuite :
    - Si la **tête touche un mur** : le jeu s'arrête.
    - Si la **tête touche un autre anneau** (sauf le premier juste derrière) : le jeu s'arrête aussi.
    - Si la **tête touche une nourriture** : elle appelle check_and_eat_food().
    - Le premier anneau (le plus proche de la tête) prend la place de la tête.
    - Le deuxième anneau prend la place du premier, et ainsi de suite.
    - Cela permet de simuler un mouvement fluide du serpent où chaque segment suit le précédent

void check_and_eat_food(Game* game)

Cette fonction :

- Lit la **case sous la tête du serpent**.
- Si c'est une nourriture (ni Empty, ni une étoile) :
    - Le serpent **ouvre la bouche**.
    - Un **nouvel anneau** est ajouté :
        - Si le serpent est vide : l'anneau devient le **premier et dernier**.
        - Sinon : le nouvel anneau devient le **premier**, l'ancien premier devient le second.
    - Le **score** est augmenté de 1 + taille du serpent.
    - Appelle remove_middle_anneau(game) pour vérifier s'il faut **supprimer un anneau central** si 3 anneaux consécutifs ont la même couleur.
    - Si on supprime alors le score diminue de -4
    - Ajoute une **nouvelle nourriture** à une position **aléatoire libre**.

bool save_game(Game* game, string filename)

Cette fonction :

- Sauvegarde la **position (**x**,** y**)**, la **direction** du serpent.
- Sauvegarde aussi la **position de toutes les nourritures**.
- Elle écrit ces informations dans un **fichier texte**.
- Elle permet à l'utilisateur de décider, via le clavier, s'il veut :
    - Continuer la partie(touche O)
    - Quitter(touche N)

void remove_middle_anneau(Game* game)

Cette fonction :

- Parcourt le serpent à partir de la tête.
- Vérifie si **trois anneaux consécutifs** ont la **même couleur**.
- Si oui, elle **supprime l'anneau du milieu**.
- Elle continue la vérification jusqu'à ce qu'il n'y ait plus de telles combinaisons.

Problèmes et difficultés rencontrés :

1. **Compréhension de la logique du jeu** : L'un des premiers défis que j'ai rencontrés a été la gestion de la tête du serpent. Au départ, je ne comprenais pas bien comment l'implémenter correctement, notamment le lien avec les anneaux du corps et leur déplacement. Ce problème m'a demandé pas mal de temps et d'essais pour être résolu.
2. **Décalage visuel** : Un autre point technique complexe a été la gestion du décalage visuel entre les différentes parties du serpent. J'ai eu besoin de l'aide de Monsieur Arthur Bernod pour mieux comprendre ce mécanisme, ce qui m'a permis de corriger l'affichage du serpent.
3. **Travail en binôme difficile** : Malheureusement, mon binôme ne s'est pas du tout investi dans le projet. Il m'a souvent donné des excuses du type "j'ai mal à la tête", "j'ai un autre projet", ou encore "j'ai un examen à préparer". Il n'a proposé aucune idée, n'a jamais demandé à voir le code, et ce n'est qu'à la toute fin qu'il a jeté un œil rapide et jugé que "c'était bon", alors que le jeu avait encore des problèmes. Cela a été une expérience exigeante, mais aussi formatrice.

Ce que j'ai appris :

Malgré les difficultés, ce projet m'a permis de développer de nombreuses compétences : gestion d'un projet en autonomie, programmation en C++ avec la bibliothèque SDL2, structuration d'un code de jeu vidéo, gestion d'événements, logique de déplacement, sauvegarde/chargement, etc.

J'ai aussi appris une leçon importante sur le travail en équipe : une collaboration efficace repose sur une implication mutuelle. Un binôme peut être une force, mais cela dépend de la motivation des deux membres. Néanmoins, je reste compréhensif face aux contraintes que mon binôme a pu rencontrer.

Je ne dirais pas que le jeu est parfait, mais j'ai fait tout ce qui était en mon pouvoir pour le rendre fonctionnel et cohérent. Ce projet a été un bon exercice de persévérance et de mise en pratique de mes apprentissages.